

Entornos de Desarrollo.

Tarea unidad 5.

1. Lectura y comprensión del enunciado.....	2
2. Tabla de sustantivos.....	2
3. Selección de clases del sistema.....	3
4. Obtención de atributos.....	3
5. Métodos de cada clase.....	5
6. Relaciones entre clases.....	6
Herencia.....	6
Composición.....	9
GreenBox compone:.....	9
EmpresaTransporte compone:.....	9
Envio compone:.....	9
Pedido compone:.....	9
Cliente compone:.....	10
Agregación.....	10
Envio agrega:.....	10
Producto agrega:.....	10
CampanyaMensual agrega:.....	10
LienaPedido agrega:.....	10
Asociación simple.....	10
Relación usar y relación validar.....	11
Diagrama final.....	12
Comentario.....	12
Anexo.....	13
Prueba de trabajo personal:.....	13

1. Lectura y comprensión del enunciado

Se trata de un sistema de gestión de ventas y comercio electrónico (e-commerce) para la venta minorista de productos ecológicos.

El sistema debe gestionar un modelo de negocio basado en campañas mensuales, lo que implica control sobre la apertura/cierre de campañas, la gestión de inventario, el procesamiento de pedidos, la logística de envíos y la interacción con usuarios externos (clientes) e internos (administrativo y almacenero).

2. Tabla de sustantivos

Sustantivo	Posible Tipo de entidad
GreenBox (Tienda)	Clase
Almacen	Clase
Cliente	Clase, Actor
Usuario	Clase, Actor
Campaña Mensual	Clase
Producto	Clase
Producto Alimentacion	Clase
Producto Cosmético	Clase
Producto Limpieza	Clase
Pedido	Clase
Línea de pedido	Clase
Marca	Clase, atributo de producto
Proveedor	Clase, atributo (lista) de marca
Tarjeta bancaria	atributo de pedido
Empleado Almacen	Clase, Actor

Empresa Transporte	Clase, Actor (declara incidencias)
Envío	Clase, atributo de Pedido
Ruta	Clase, atributo de envío
Incidencia Envío	Clase, atributo (lista) de envío.
Empleado Administrativo	Clase, Actor
Entidad Bancaria	Clase

3. Selección de clases del sistema

Las clases que creo que deben existir son:

GreenBox (Tienda), Almacén, Cliente, Campaña Mensual, Producto, Producto Alimentación, Producto Cosmético, Producto Limpieza, Pedido, Línea de pedido, Marca, Proveedor, Empleado Almacén, Empresa Transporte, Envío, Ruta, Incidencia Envío, Empleado Administrativo, Entidad Bancaria.

Se ha de tener en cuenta que esta lista es fruto del análisis, una vez en el diseño se amplía con enumeraciones y ciertas estructuras de herencia, lo que hace que al final vaya a haber más entidades de las aquí listadas.

4. Obtención de atributos

Aquí se me plantea una duda muy importante, ya que en UML ciertas relaciones implican atributos, que no han de declararse otra vez como tales, y por tanto ignoro si debo incluir aquí que la tienda tiene un almacén, o eso queda, como se supone, implícito por la composición que relacionará las dos entidades.

Dado que esto es fase de análisis, decido expresarlo.

Clase	Atributos
GreenBox (Tienda)	- almacen, Almacen - clientes, List Cliente - proveedores. List Proveedor - empleado almacén - empleado administrativo - transportista, EmpresaTransporte - campanyaActual, CampanyaMensual
Almacén	- Productos, List Producto
Cliente	- nombre, String - CIF, String - domicilio fiscal, String - pedidos, List Pedido
CampanyaMensual	- fecha inicio, Date - fecha fin, Date - productos, List Producto
Producto	- nombre, String - descripción, String - categoría, String - precio, double - stock, int - marca, Marca
ProductoAlimentacion	- peso - perecedero
ProductoCosmético	- tipo de piel
ProductoLimpieza	- formato
Pedido	- cliente - tarjeta bancaria - líneas de pedido - preparado - envio
LineaPedido	- pedido - número
Marca	- nombre - CIF - Domicilio fiscal - proveedores
Proveedor	- nombre

	- CIF - Domicilio fiscal - marcas
EmpleadoAlmacen	
EmpresaTransporte	- Rutas
Envío	- fecha - ruta - incidencias
Incidencia Envío	- fecha - descripción
Ruta	- zona geográfica - días
Empleado Administrativo	

5. Métodos de cada clase

En esta lista no voy a incluir los getters/setter o constructores. Esta lista se ha mantenido actualizada según se hacía el diseño, a diferencia de los puntos anteriores. Las clases que no tienen métodos que no sean de los excluidos no aparecerán en la lista. Defino la firma con estilo java.

Clase	Métodos
Almacen	public Producto[] consultarProductosDisponibles(java.util.Date hoy)
Cliente	public void realizaPedido(Pedido pedido)
	public Pedido[] consultaPedidos()
	public boolean cancelaPedidoPendiente(Pedido pedido)
	public void modificaDatosPersonales(String email, String direccion, String nombreCompleto)

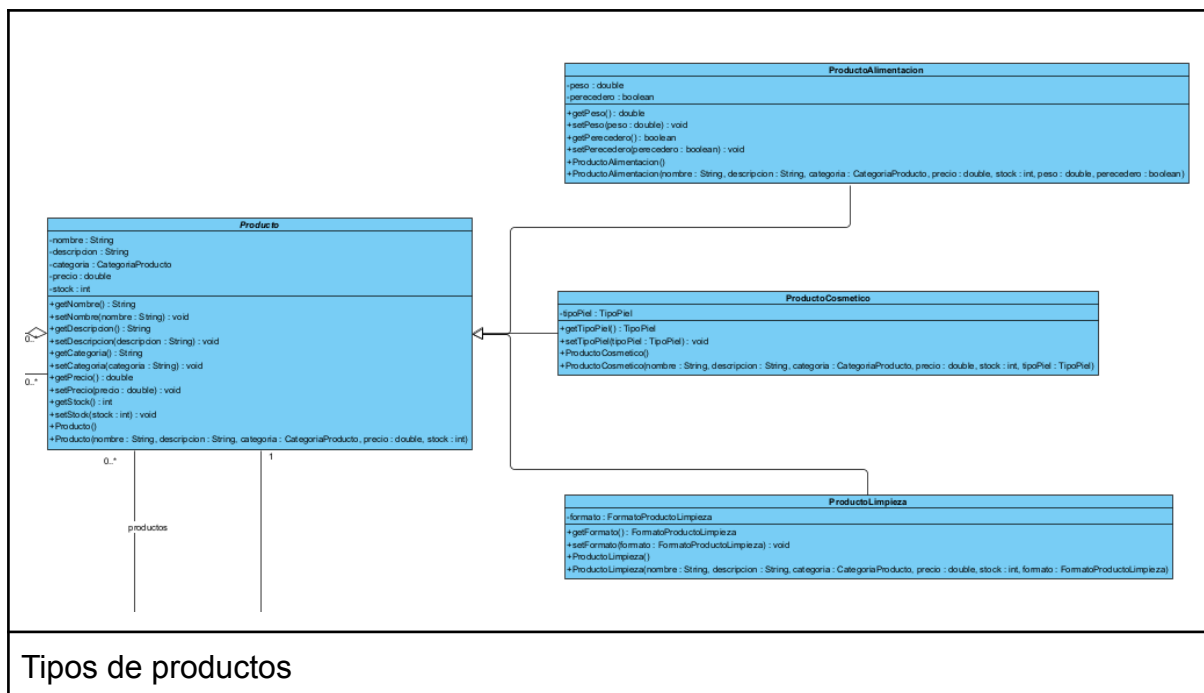
Empleado	public Pedido[] revisaPedidosPendientes()
EmpleadoAdministrativo	public void abrirCampanya(CampanyaMensual campanya)
	public void cerrarCampanyaActual()
EmpleadoAlmacen	public void preparaPedido(Pedido pedido)
EmpresaTransporte	public void nuevaIncidenciaEnvio(java.util.Date fecha, String descripcion, Envio envio)
EntidadBancaria	public boolean validaTarjeta(int numTarjeta)
	public boolean cobra(double cantidad, int numTarjeta)
GreenBox	public void registraNuevoCliente(Cliente cliente)
	public Producto[] productosDisponiblesAlmacen()
	public Pedido[] pedidosPendientes()
	public void enviaPedido(Pedido pedido, EmpresaTransporte empresa)
	public void productosDisponiblesCampanya()
Usuario	public Producto[] consultaProductosCampanyaActual(String porNombre)
	public void darDeAltaNuevoCliente(String email, String nombre, String direccion)

6. Relaciones entre clases

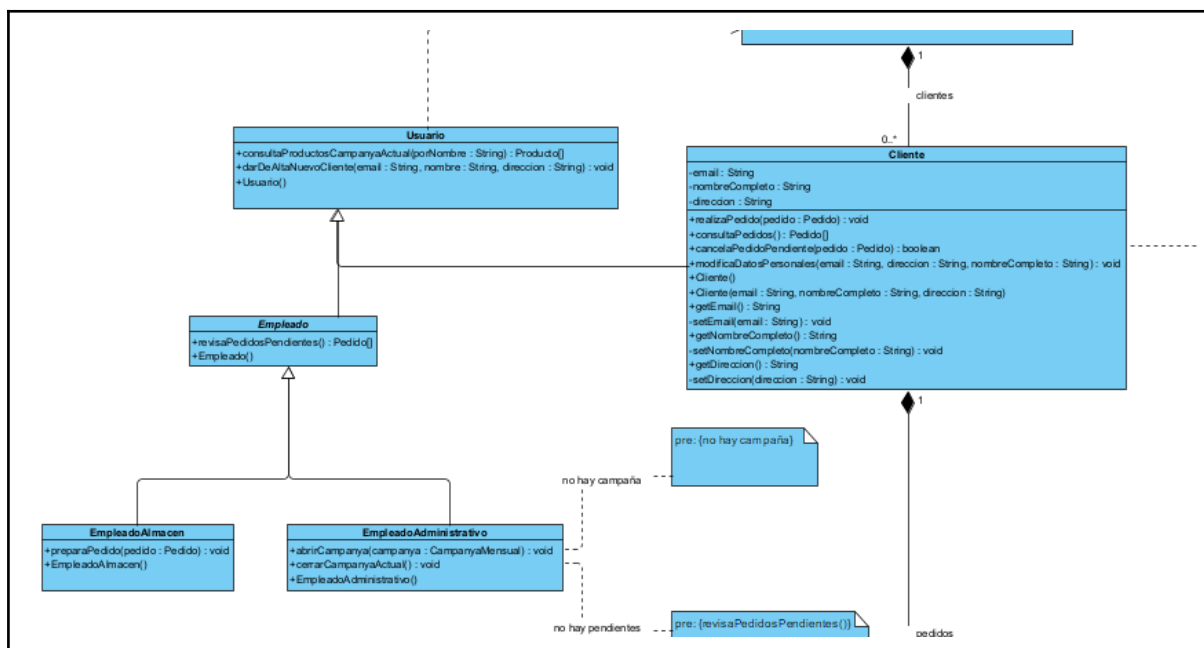
Estas relaciones se han mantenido también actualizadas, y es por tanto fruto del esquema final entregado.

Herencia

En general, se ha tratado de hacer que cada clase que no se pueda instanciar ya que requiera necesariamente un tipo sea abstracta, por ejemplo Producto.

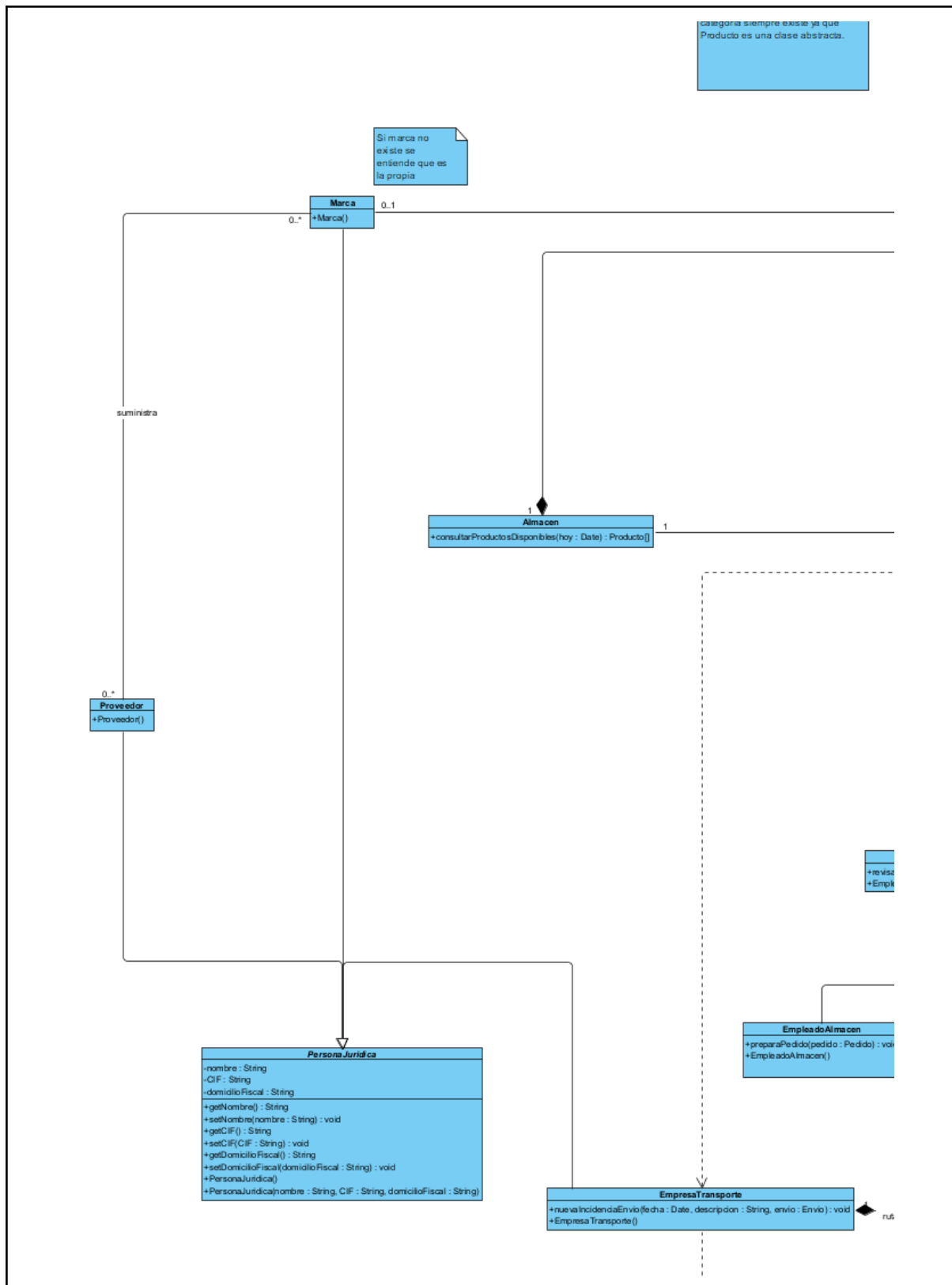


Tipos de productos



Tipos de usuarios, con diferenciación entre usuarios empleados y usuarios clientes, se ha hecho así para facilitar la separación de roles y agrupar acciones en común, la búsqueda de pedidos pendientes, que debe poder hacer tanto el empleado de almacén como el de administración.

Podría hacerse en dos ramas completamente diferenciadas, pero permito que un empleado busque productos y se cree un usuario si lo desea.



Se usa la herencia desde persona jurídica para todas las entidades que se han detectado que lo son, no implementan funcionalidad común, pero sí comparten datos.

Composición

La composición implica que la entidad secundaria sólo tiene sentido mientras exista la primaria.

GreenBox compone:

1 - 0..1 Campaña (una campaña solo existe para nuestra tienda y además es única o nula)

1 - 1 Almacén, de nuevo, almacén solo tiene sentido con GreenBox, y es único y existe siempre.

1 - 0..n Cliente, Greenbox tiene clientes que solo existen si greenbox existe.

EmpresaTransporte compone:

1 - 0..n Rutas, la empresa define y maneja sus rutas

Envio compone:

1 - 0..n IncidenciaEnvio, las incidencias solo existen para un envío dado

Pedido compone:

1 - 1..n LineaPedido, en efecto, las líneas de pedido solo existen en pedido y además ha de haber al menos una

1 - 1 Envio, solo definimos un envio para cada pedido, esto podría hacerse diferente si asumimos que un envío ha fallado, por ejemplo, pero no se indica nada en la tarea sobre este escenario.

Cliente compone:

1 - 0..n Pedidos, un pedido sólo existe porque lo hace un cliente.

Agregación

Envio agrega:

0..n - 1 Ruta, es decir, muchos envíos pueden tener la misma ruta, y siempre ha de haber una en cada envío. la ruta existe al margen del envío.

Producto agrega:

0..n - 0..1 Marca, en efecto, todo producto tiene una marca o ninguna, si es la propia. La marca existe al margen del producto.

CampanyaMensual agrega:

1 - 0..n Producto, una campaña consta de productos, que existen al margen de la campaña.

LienaPedido agrega:

0..n - 1 Producto, una línea de pedido tiene un producto asociado, que existe al margen de la línea.

Asociación simple

Así expresamos cuando una clase conoce a otra en algún grado, pero no se puede relacionar de un modo jerárquico claro.

Proveedor (0..n) - Marca (0..n)

En efecto, cada marca puede ser distribuida por varios proveedores y un proveedor puede distribuir varias marcas.

Relación usar y relación validar

Si bien es cierto que con tan solo nombrar una clase como parte de la firma de un método de otra ya es una declaración implícita de que una clase usa a otra, en algunos casos eso no puede expresarse o no es necesario, y he optado por expresar esos usa en diferentes partes.

Así GreenBox usa a la entidad bancaria y a la empresa de transporte, y es usada por todos los usuarios (que recordemos que agrupa por herencia a empleados y usuarios anónimos y clientes)

A su vez, la empresa de transporte usa el envío para añadir una incidencia.

Las validaciones están expresadas como notas enlazadas al método cuya ejecución depende de la validación, al parecer es así como debe hacerse.

Por último señalar que creo que a través de composición, usage y agregación, las acciones declaradas pueden delegarse implícitamente (el usuario que busca libros lo hace a través de GreenBox), aunque esa acción en la clase a la que se delega no siempre está implementada, he tratado de expresarla en aquellos casos en los que funcionalmente tenía, en mi opinión, más sentido. Como por ejemplo en la propia GreenBox, clase que solo tiene atributos a través de sus relaciones con las demás y que sirve como un orquestador de muchas peticiones.

El código resultante es, en mi opinión, muy deficiente, ya que ninguna de las relaciones expresadas se ha tenido en cuenta a la hora de asignar y definir atributos. Al menos compila :D

Anexo

Prueba de trabajo personal:

The screenshot displays a web application interface for 'Entorno de desarrollo (DAW)'. The interface is divided into several sections:

- General:** A central area with a blue header and a large image of a person at a computer. Below the image, there is a description of the module and a list of communication channels (Email, Facebook, Twitter, YouTube, etc.).
- Calendar:** A section on the right showing a calendar for March 2026. It includes a table of dates and a list of events.
- Personas:** A section on the right showing a list of participants and their status.
- UML Diagram:** A UML diagram on the right side of the screen, showing a class hierarchy and relationships between classes.

Below the screenshot, there is a caption:

Adjunto pantallazo del proceso de diseño en VP incluyendo login en la aplicación del centro